

Optimization for Deep Learning

Adam, K-FAC, and the Modern Story

Dr. Robert Hildebrand

Grado Department of Industrial & Systems Engineering
Virginia Tech

Roadmap

From Yesterday's Theory to Today's Practice

The DL Training Problem

ADMM is not Adam

Per-coordinate Adaptive Methods

When Adam Fails, and Why

Second-Order Revival

Loss-Landscape Phenomena

Wrap-Up

Where we are

What we have so far.

- Convex theory: GD, CG, momentum, Nesterov. Sharp rates, matching lower bounds.
- SGD with proofs: $O(1/\sqrt{K})$ convex case, $O(1/k)$ strongly convex, mini-batch reduces variance by $1/b$.
- SVM with kernels: dual, support vectors, Mercer's condition.

What changes when we go to deep learning.

- **Non-convex.** The loss surface has many local minima, saddles, plateaus. Convergence guarantees become much weaker (gradient norm $\rightarrow 0$, not function value to global optimum).
- **Very high dimension.** $p \sim 10^6$ for vision models, $\sim 10^{11}$ for foundation models. Hessian-vector products are tractable, full Hessians are not.
- **Heterogeneous parameter scales.** Weights in different layers have wildly different gradient magnitudes. A single learning rate doesn't fit all.
- **Noisy gradients.** Mini-batch noise + dropout + data augmentation. The SGD variance term σ^2 is large.
- **Non-stationary curvature.** The local Lipschitz constant L changes during training.

Today. The optimizers practitioners actually use — AdaGrad, RMSProp, Adam, AdamW, K-FAC, Shampoo, Sophia — and what's known about why they work.

Theme. Every modern optimizer is one of (i) SGD with smarter step sizes, (ii) SGD with smarter momentum, or (iii) SGD with a curvature approximation. We'll see all three.

Today's roadmap

1. **The DL training problem.** What's hard, what's not, what makes the noise structure different from textbook SGD.
2. **ADMM $\neq$ Adam.** Quick disambiguation — two famous algorithms with confusingly similar names.
3. **Per-coordinate adaptive methods.** AdaGrad $\rightarrow$ RMSProp $\rightarrow$ Adam $\rightarrow$ AdamW $\rightarrow$ Lion. Each one is a small modification of the previous.
4. **Why Adam works (and where it doesn't).** Bias correction, the Reddi–Kale–Kumar non-convergence counterexample, AMSGrad fix.
5. **Second-order revival.** K-FAC, Shampoo, Sophia — the modern crop of curvature-aware optimizers.
6. **Loss-landscape phenomena.** Saddle points dominate; “edge of stability”; large-batch training; lottery tickets.
7. **Wrap-up + bridge to RLHF.** Where today's optimizers leave off and the next lecture picks up.

Reading today's content. Bottou, Curtis & Nocedal (2018), *Optimization Methods for Large-Scale Machine Learning*, SIAM Review 60(2): the textbook survey. Goh (2017), *Why Momentum Really Works*, Distill: the geometric picture. Kingma & Ba (2015), *Adam: A Method for Stochastic Optimization*, ICLR.

Roadmap

From Yesterday's Theory to Today's Practice

The DL Training Problem

ADMM is not Adam

Per-coordinate Adaptive Methods

When Adam Fails, and Why

Second-Order Revival

Loss-Landscape Phenomena

Wrap-Up

The deep learning training problem

Same empirical-risk template, but h_θ is now a deep neural net:

$$F(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(h_\theta(\mathbf{x}_i), y_i) + R(\theta).$$

Five things break textbook intuition.

Issue	Convex world	Deep learning
Convexity	Convex, often strongly	Non-convex, negative curvature
Dimension	$p \leq 10^4$	$p \in [10^6, 10^{11}]$
Gradient noise	σ^2 small	σ^2 large (dropout, aug)
Per-coord scale	uniform	varies $10^4 \times$ across layers
Curvature	L, μ global	local L changes $\sim 100 \times$

What survives. Mini-batch SGD, the unbiased-gradient framework, momentum, step-size schedules.

What's new. Per-coordinate steps (AdaGrad family), cheap curvature (K-FAC, Shampoo), saddle-robust updates.

Neural networks: the function class

Bishop & Bishop 2024, §6.2–6.3.

Definition. A neural network is a composition of affine maps and elementwise non-linearities:

$$h_{\theta}(\mathbf{x}) = A_L \circ \sigma_{L-1} \circ A_{L-1} \circ \cdots \circ \sigma_1 \circ A_1(\mathbf{x}), \quad A_{\ell}(\mathbf{z}) = W_{\ell} \mathbf{z} + \mathbf{b}_{\ell}.$$

Forward pass. $\mathbf{a}_0 = \mathbf{x}$; for $\ell = 1, \dots, L$:

$$\mathbf{z}_{\ell} = W_{\ell} \mathbf{a}_{\ell-1} + \mathbf{b}_{\ell}, \quad \mathbf{a}_{\ell} = \sigma_{\ell}(\mathbf{z}_{\ell}).$$

Output: $h_{\theta}(\mathbf{x}) = \mathbf{a}_L$. Parameters: $\theta = \{W_{\ell}, \mathbf{b}_{\ell}\}_{\ell=1}^L$. Foundation models: $L \sim 100$, $|\theta| \sim 10^9$ – 10^{11} .

Optimization problem unchanged.

$$\min_{\theta} F(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(h_{\theta}(\mathbf{x}_i), y_i) + R(\theta).$$

We feed h_{θ} and ∇F to SGD / Adam / K-FAC. The new ingredients: (i) a way to draw the network, (ii) the activation σ choices, (iii) the theorems that say this class is rich enough, (iv) backpropagation for the gradient.

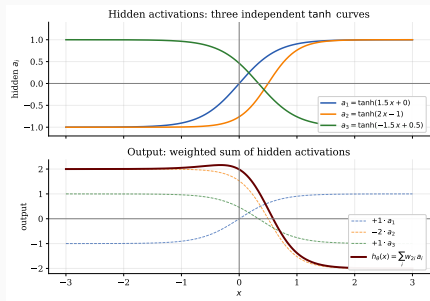
A worked example: 1-input, 3-hidden, 1-output

Architecture. One input x , three hidden units with tanh activation, one linear output:

$$h_{\theta}(x) = w_{2,1} \tanh(w_{1,1}x + b_{1,1}) + w_{2,2} \tanh(w_{1,2}x + b_{1,2}) + w_{2,3} \tanh(w_{1,3}x + b_{1,3}) + b_2.$$

Pick parameters by hand.

$$W_1 = \begin{pmatrix} 1.5 \\ 2.0 \\ -1.5 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 0 \\ -1 \\ 0.5 \end{pmatrix}, \quad \mathbf{w}_2 = \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix}, \quad b_2 = 0 \implies h_{\theta}(x) = \tanh(1.5x) - 2 \tanh(2x-1) + \tanh(-1.5x+0.5).$$

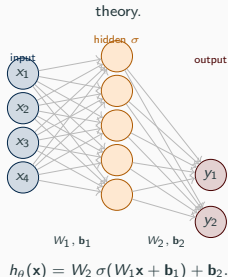


Top: the three hidden activations — each a single tanh curve, shifted and scaled. Bottom: their weighted contributions (dashed) sum to the output $h_{\theta}(x)$ (maroon). **Three S-curves combine into a non-trivial function with a peak, a valley, and a tail.** Adding more hidden units lets us approximate any continuous function (next slides).

What a neural network looks like

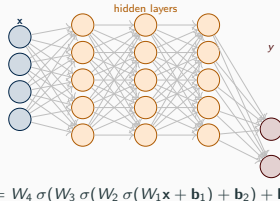
Two-layer network ($L = 2$).

One hidden layer, the workhorse of universal-approximation



Deep network ($L \geq 3$).

Each hidden layer applies its own affine map and non-linearity.

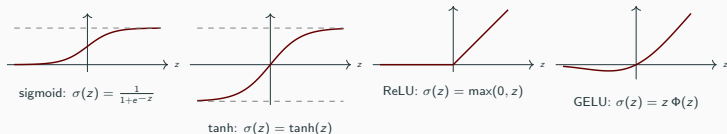


Reading the picture. Each circle is a scalar (a coordinate of $\mathbf{a}_{\ell}$). Each arrow is one entry of W_{ℓ} . Bias $\mathbf{b}_{\ell}$ is omitted from the drawing for clarity. The forward pass moves left to right.

Why the depth helps in practice. A wide 2-layer net can match a deep one in expressive power, but typically needs *exponentially more* hidden units to do so (Telgarsky 2016). *Depth is parameter-efficient expressiveness.*

Activation functions σ

Why we need them. Without a non-linearity, $A_L \circ \dots \circ A_1$ collapses to a single affine map $\mathbf{x} \mapsto W\mathbf{x} + \mathbf{b}$ — linear regression, no matter how deep. The activation σ is what makes depth useful.



Activation	Used in	Property
sigmoid $\frac{1}{1+e^{-z}}$	classical 2-layer nets, output gates	saturates; vanishing gradients in deep nets
tanh $\frac{e^z - e^{-z}}{e^z + e^{-z}}$	RNNs, classical hidden layers	zero-centered version of sigmoid
ReLU $\max(0, z)$	most modern feed-forward nets, CNNs	cheap, no saturation for $z > 0$, sparse
GELU $z\Phi(z)$	Transformers (BERT, GPT)	smooth ReLU; differentiable everywhere
softmax (output)	classification heads	normalizes vector to probability simplex

Practical default in 2025: ReLU for vision/CNN hidden layers, GELU for transformer hidden layers, softmax for classification outputs.

Universal approximation: any continuous function, in principle

Bishop & Bishop 2024, §6.2.3.

The original theorem.

Theorem (Cybenko 1989, Math. Control Signals Systems 2)

Let σ be a continuous sigmoidal function (e.g., $\sigma(z) = \frac{1}{1+e^{-z}}$). Then for every continuous $f : [0, 1]^n \rightarrow \mathbb{R}$ and every $\varepsilon > 0$, there exist $N \in \mathbb{N}$, vectors $\mathbf{w}_j \in \mathbb{R}^n$, and constants $\alpha_j, \theta_j \in \mathbb{R}$ such that the 2-layer network

$$F(\mathbf{x}) = \sum_{j=1}^N \alpha_j \sigma(\mathbf{w}_j^\top \mathbf{x} + \theta_j)$$

satisfies $\sup_{\mathbf{x} \in [0, 1]^n} |F(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$.

Generalizations.

- Any non-polynomial activation works [Hornik–Stinchcombe–White 1989; Leshno–Lin–Pinkus–Schocken 1993, *Neural Networks* 6]: a 2-layer network with activation σ is a universal approximator if and only if σ is non-polynomial. *This is what justifies ReLU, GELU, etc. — not just sigmoidal.*
- Depth is exponentially more efficient than width [Telgarsky 2016, COLT, “Benefits of depth in neural networks”]: there exist functions on $[0, 1]^d$ that a deep ReLU network of depth $\Theta(L)$ and constant width represents, but any ReLU network of depth $o(L/\log L)$ needs $\Omega(2^L)$ width to approximate within $\sim$ constant error.

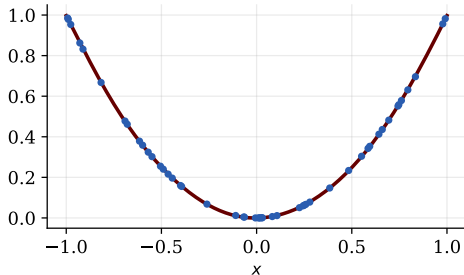
The catch (and why we still need optimization)

- Universal approximation theorems are **existence statements**: a good network *exists*. They say nothing about whether SGD / Adam can *find* it.
- They also don’t say how big N needs to be — in worst case, exponential in dimension (Barron 1993).
- Deep learning works in practice because optimization, generalization, and architecture choices conspire well — not because universal approximation alone guarantees it.

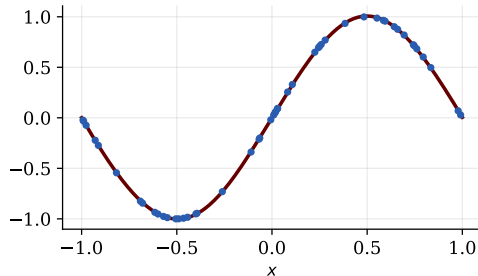
Universal approximation, visualized: a 2-layer net fits four functions

Universal approximation: a 2-layer net fits four very different functions

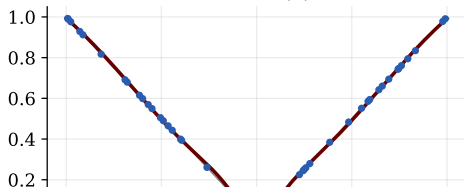
(a) $f(x) = x^2$



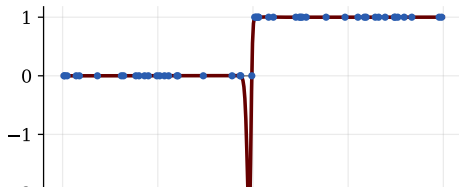
(b) $f(x) = \sin(\pi x)$



(c) $f(x) = |x|$

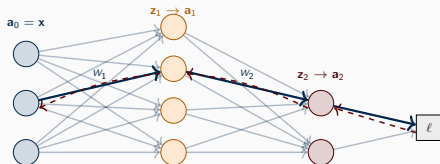


(d) $f(x) = H(x)$



Backpropagation: forward and backward through the network

Bishop & Bishop 2024, §8.1.2.



forward (solid blue): compute $\mathbf{z}_\ell, \mathbf{a}_\ell$

backward (dashed maroon): compute $\delta_\ell, \nabla_{W_\ell} \ell, \nabla_{\mathbf{b}_\ell} \ell$

Forward pass (left $\rightarrow$ right): store $\mathbf{z}_\ell, \mathbf{a}_\ell$.

Backward pass (right $\rightarrow$ left): seed $\delta_L = \nabla_{\mathbf{a}_L} \ell$,
then for $\ell = L, \dots, 1$:

$$\mathbf{a}_0 = \mathbf{x}, \mathbf{z}_\ell = W_\ell \mathbf{a}_{\ell-1} + \mathbf{b}_\ell, \mathbf{a}_\ell = \sigma_\ell(\mathbf{z}_\ell).$$

$$\nabla_{W_\ell} \ell = \delta_\ell \mathbf{a}_{\ell-1}^\top, \nabla_{\mathbf{b}_\ell} \ell = \delta_\ell, \delta_{\ell-1} = W_\ell^\top (\delta_\ell \odot \sigma'_\ell(\mathbf{z}_\ell)).$$

Cost. One forward + one backward = full $\nabla_{\theta} \ell$. Both passes are $O(\sum_\ell n_\ell n_{\ell-1})$ flops — linear in parameters, regardless of network depth. Activations from the forward pass must be *stored* for the backward pass; that's the GPU-memory bottleneck.

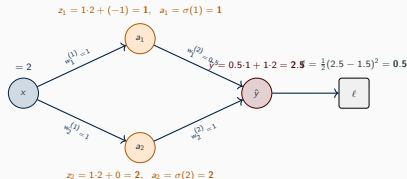
In code. Modern frameworks (PyTorch, JAX) record a *computation graph* during the forward pass and run autodiff to do backprop automatically.

Backprop walkthrough (1 of 2): the forward pass

Setup. A 1-input, 2-hidden, 1-output network with ReLU activation $\sigma(z) = \max(0, z)$, squared loss.

Inputs: $x = 2$, $y = 1.5$ (target). **Parameters** (all chosen for clean arithmetic):

$$w_1^{(1)} = 1, \quad b_1^{(1)} = -1, \quad w_2^{(1)} = 1, \quad b_2^{(1)} = 0, \quad w_1^{(2)} = 0.5, \quad w_2^{(2)} = 1, \quad b^{(2)} = 0.$$



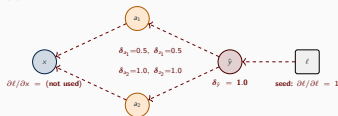
Forward computations, layer by layer.

Step	Formula	Substitute	Value
1. pre-activation, hidden 1	$z_1 = w_1^{(1)} x + b_1^{(1)}$	$1 \cdot 2 + (-1)$	1
2. activation, hidden 1	$a_1 = \sigma(z_1) = \max(0, z_1)$	$\max(0, 1)$	1
3. pre-activation, hidden 2	$z_2 = w_2^{(1)} x + b_2^{(1)}$	$1 \cdot 2 + 0$	2
4. activation, hidden 2	$a_2 = \sigma(z_2)$	$\max(0, 2)$	2
5. output (linear)	$\hat{y} = w_1^{(2)} a_1 + w_2^{(2)} a_2 + b^{(2)}$	$0.5 \cdot 1 + 1 \cdot 2 + 0$	2.5
6. loss	$\ell = \frac{1}{2}(\hat{y} - y)^2$	$\frac{1}{2}(2.5 - 1.5)^2$	0.5

We save the activations $z_1, a_1, z_2, a_2, \hat{y}$. The backward pass needs them. (Next slide.)

Backprop walkthrough (2 of 2): the backward pass

Same network, same numbers; we now propagate gradients right to left. Read each row as one chain-rule link.



Backward computations, link by link.

Step	Chain-rule formula	Substitute	Value
1. seed at output	$\partial \ell / \partial \hat{y} = \hat{y} - y$	$2.5 - 1.5$	1.0
Output layer parameters:			
2. $\partial \ell / \partial w_1^{(2)}$	$= (\partial \ell / \partial \hat{y}) \cdot a_1$	$1.0 \cdot 1$	1.0
3. $\partial \ell / \partial w_2^{(2)}$	$= (\partial \ell / \partial \hat{y}) \cdot a_2$	$1.0 \cdot 2$	2.0
4. $\partial \ell / \partial b^{(2)}$	$= \partial \ell / \partial \hat{y}$	1.0	1.0
Propagate gradient to hidden activations a_1, a_2 :			
5. $\partial \ell / \partial a_1$	$= (\partial \ell / \partial \hat{y}) \cdot w_1^{(2)}$	$1.0 \cdot 0.5$	0.5
6. $\partial \ell / \partial a_2$	$= (\partial \ell / \partial \hat{y}) \cdot w_2^{(2)}$	$1.0 \cdot 1$	1.0
Through the activation: multiply by $\sigma'(z) = \mathcal{H}[z > 0]$. Both $z_1, z_2 > 0$ so $\sigma' = 1$:			
7. $\partial \ell / \partial z_1$	$= (\partial \ell / \partial a_1) \cdot \sigma'(z_1)$	$0.5 \cdot 1$	0.5
8. $\partial \ell / \partial z_2$	$= (\partial \ell / \partial a_2) \cdot \sigma'(z_2)$	$1.0 \cdot 1$	1.0
Hidden layer parameters:			
9. $\partial \ell / \partial w_1^{(1)}$	$= (\partial \ell / \partial z_1) \cdot x$	$0.5 \cdot 2$	1.0
10. $\partial \ell / \partial b_1^{(1)}$	$= \partial \ell / \partial z_1$	0.5	0.5
11. $\partial \ell / \partial w_2^{(1)}$	$= (\partial \ell / \partial z_2) \cdot x$	$1.0 \cdot 2$	2.0
12. $\partial \ell / \partial b_2^{(1)}$	$= \partial \ell / \partial z_2$	1.0	1.0

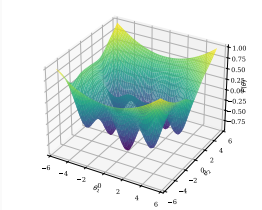
Gradient at every parameter.

$$\nabla_{W^{(1)}} \ell = \begin{pmatrix} 1.0 \\ 2.0 \end{pmatrix}, \quad \nabla_{b^{(1)}} \ell = \begin{pmatrix} 0.5 \\ 1.0 \end{pmatrix}, \quad \nabla_{W^{(2)}} \ell = (1.0, 2.0), \quad \nabla_{b^{(2)}} \ell = 1.0.$$

Two things to notice. (1) Each $\partial \ell / \partial w^{(l)}$ multiplies the upstream δ by the input that fed that weight (forward-pass activation). (2) A gradient passes through a ReLU only when the unit was active in the forward pass — if $z_1 < 0$ then $\sigma'(z_1) = 0$ and the gradient is killed at that node. This is the local mechanism that produces sparse gradients in deep ReLU nets.

The deep-learning loss landscape: a rugged sea of local minima

A typical deep-learning loss surface: many basins of similar depth



What you're looking at. A typical 2-D slice through a deep-network loss surface.

Many basins, all of similar depth. The loss is **deeply non-convex**: where convex theory promises a single global minimum, deep learning hands you *thousands* of nearly-equivalent ones.

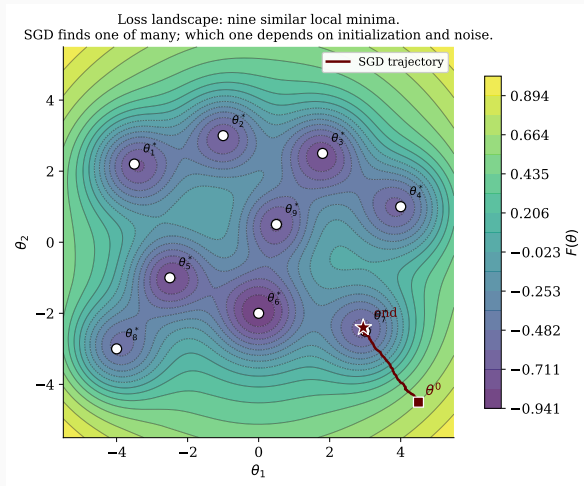
Insight

The textbook worry — “GD gets stuck in a bad local minimum” — is mostly the wrong worry for deep learning. **Most local minima have similar training loss**; what differs between them is **generalization**. Which one your optimizer finds matters far more than whether it finds one.

Citations for the “similar-depth minima” claim.

- Choromanska, Henaff, Mathieu, Ben Arous, LeCun (2015), *The Loss Surfaces of Multilayer Networks*, AISTATS — spin-glass analysis: values of local minima concentrate in a narrow band that shrinks with network width.
- Dauphin, Pascanu, Gulcehre, Cho, Ganguli, Bengio (2014), NeurIPS — empirical: local minima cluster at low loss, saddles dominate everything else.
- Garipov, Izmailov, Podoprikin, Vetrov, Wilson (2018), *Loss Surfaces, Mode Connectivity, and Fast Ensembling*, NeurIPS — mode connectivity: pairs of SGD solutions are connected by curves of *nearly constant loss*. Stronger than “similar depth”.
- Draxler, Veschgini, Salmhofer, Hamprecht (2018), *Essentially No Barriers in Neural Network Energy Landscape*, ICML — concurrent confirmation of mode connectivity.
- Frankle, Dziugaite, Roy, Carbin (2020), *Linear Mode Connectivity and the Lottery Ticket Hypothesis*, ICML — mode connectivity holds linearly after early training.

Loss landscape: many similar minima, one trajectory



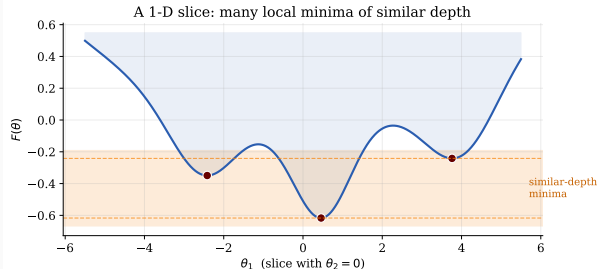
The contour view. Same surface as the previous slide, top-down.

- **White circles:** nine local minima $\theta_1^*, \dots, \theta_9^*$. All within $\sim 10\%$ of each other in loss.
- **Maroon trajectory:** SGD started at the bottom-right, found one specific basin.
- **Different start** or **different noise** would have ended up in a different basin.

What this picture says.

- Convergence guarantees in deep learning are about $\|\nabla F\| \rightarrow 0$, not $F \rightarrow F^*$. The loss-floor itself is

1-D slice: many similar-depth minima at a glance



Reading the slice. A line through the 2-D landscape (held $\theta_2 = 0$). Several local minima (maroon dots), all within a narrow loss band (orange shading). Plain GD with a small enough η converges to whichever minimum it starts nearest to. Adding stochasticity (SGD) lets the iterate hop between basins and find one of the deeper ones — a feature, not a bug.

Take-away

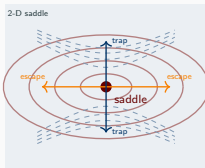
Deep-learning loss surfaces are **highly non-convex** but **benign**: many minima, similar depths. Optimization algorithms compete on *which* minimum (and therefore which generalization) they find, not whether they find one.

Saddle points dominate in high dimensions

The classical worry: local minima. In low dimensions, optimization on non-convex f gets stuck in bad local minima. In high dimensions, this picture is wrong.

Theorem (Dauphin, Pascanu, Gulcehre, Cho, Ganguli, Bengio 2014, NeurIPS, “Identifying and attacking the saddle point problem in high-dimensional non-convex optimization”)
For a random Gaussian f in $\mathbb{R}^p$, a critical point at level α above the global minimum has (in expectation) approximately $\alpha\sqrt{p}$ negative-curvature directions. Generic critical points in high dimension are saddle points, not local minima.

Consequence. Plain GD slows to a crawl near saddles (gradient is small but progress is blocked along negative-curvature directions). Methods that exploit *any* stochasticity — SGD, mini-batch SGD, momentum — escape saddles in expectation in $\tilde{O}(1/\varepsilon^2)$ steps [Ge–Huang–Jin–Yuan 2015].



Picture. In $\mathbb{R}^2$ a saddle has 1 negative-curvature direction; in $\mathbb{R}^p$ it can have $\sqrt{p}$ of them. Stochastic noise picks them up; deterministic GD does not.

Takeaway. The textbook “GD gets stuck in local minima” worry is the wrong worry for deep learning. The real problem is escaping saddles efficiently. SGD’s variance is a feature, not a bug — it’s what frees the iterate from negative-curvature traps.

Roadmap

From Yesterday's Theory to Today's Practice

The DL Training Problem

ADMM is not Adam

Per-coordinate Adaptive Methods

When Adam Fails, and Why

Second-Order Revival

Loss-Landscape Phenomena

Wrap-Up

ADMM $\neq$ Adam (one slide of disambiguation)

Two famous algorithms whose names sound alike. They are unrelated.

ADMM [Glowinski–Marrocco 1975; Boyd et al. 2010]

Solves: composite split problems

$$\min_{\mathbf{x}, \mathbf{z}} f(\mathbf{x}) + g(\mathbf{z}) \text{ s.t. } A\mathbf{x} + B\mathbf{z} = \mathbf{c}.$$

Update: alternate $\mathbf{x}, \mathbf{z}, \mathbf{u}$ minimization on the augmented Lagrangian.

Used for: distributed and federated

optimization, proximal methods, imaging.

Mathematically: operator splitting on the dual.

Adam [Kingma & Ba 2015]

Solves: stochastic non-convex deep-learning training.

Update:

$$\mathbf{m}^{k+1} = \beta_1 \mathbf{m}^k + (1 - \beta_1) \mathbf{g}_k,$$

$$\mathbf{v}^{k+1} = \beta_2 \mathbf{v}^k + (1 - \beta_2) \mathbf{g}_k^2,$$

$$\theta^{k+1} = \theta^k - \eta \hat{\mathbf{m}}^{k+1} / (\sqrt{\hat{\mathbf{v}}^{k+1}} + \epsilon).$$

Used for: training neural networks (default in PyTorch/JAX).

Mathematically: per-coordinate adaptive SGD with two EMAs.

One letter different, different algorithm, motivation, and use case. ADMM lives in the splitting half of Lecture 4; the rest of today is Adam and descendants.

Roadmap

From Yesterday's Theory to Today's Practice

The DL Training Problem

ADMM is not Adam

Per-coordinate Adaptive Methods

When Adam Fails, and Why

Second-Order Revival

Loss-Landscape Phenomena

Wrap-Up

gradient descent is memoryless

The villain. Plain GD on $f(p) = p^2$. Gradient $g = 2p$. Run with $\eta = 0.1$ from $p_0 = 10$.
Update rule: $p_{\text{new}} = p - \eta g = p - 0.2p = 0.8p$.

k	p_k	$g = 2p_k$	$\Delta = -\eta g$	p_{k+1}
0	10.000	20.000	-2.000	8.000
1	8.000	16.000	-1.600	6.400
2	6.400	12.800	-1.280	5.120
3	5.120	10.240	-1.024	4.096
4	4.096	8.192	-0.819	3.277

Look at the Δ column. The step shrinks at every iteration. As p approaches the minimum, the gradient shrinks and the algorithm *decelerates*. After 5 steps we're still at 3.28 — a crawl.

Diagnosis. GD has zero memory of past steps. Each iteration starts from scratch. *We need a way to carry momentum from one step to the next.*

Solution 1: momentum gives memory

Add a velocity that remembers. Initialize $v_0 = 0$. At each step:

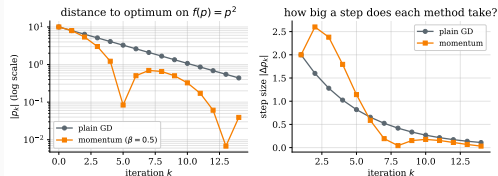
$$v_k = \beta v_{k-1} + g_k, \quad p_{k+1} = p_k - \eta v_k.$$

With $\beta = 0.5$, the velocity is half last iteration's plus the new gradient.

k	p_k	$g_k = 2p_k$	$v_k = 0.5v_{k-1} + g_k$	$\Delta = -\eta v_k$	p_{k+1}
0	10.000	20.000	$0.5 \cdot 0 + 20.0 = 20.000$	-2.000	8.000
1	8.000	16.000	$0.5 \cdot 20 + 16 = 26.000$	-2.600	5.400
2	5.400	10.800	$0.5 \cdot 26 + 10.8 = 23.800$	-2.380	3.020
3	3.020	6.040	$0.5 \cdot 23.8 + 6.04 = 17.940$	-1.794	1.226
4	1.226	2.452	$0.5 \cdot 17.94 + 2.452 = 11.422$	-1.142	0.084

Showdown. After 5 steps, plain GD: $p \approx 3.28$. Momentum: $p \approx 0.084$. $\sim 40\times$ closer.

Plain GD decelerates as the gradient shrinks; momentum keeps cruising.



Left: $|p_k|$ on a log scale — momentum's curve drops faster and stays steep. Right: the per-iteration step size $|\Delta p_k|$. Plain GD's step shrinks geometrically with p ; momentum's step grows for several iterations as the velocity accumulates, then decays smoothly.

one global learning rate is wrong

The new villain. An unbalanced 2-D function:

$$F(p_0, p_1) = 50 p_0^2 + p_1^2, \quad \nabla F = (100 p_0, 2 p_1).$$

The gradient w.r.t. p_0 is 50× larger than w.r.t. p_1 . p_0 is sensitive (a small step sends it flying); p_1 is stubborn (it barely moves).

What does plain GD do? To keep p_0 from blowing up, we're forced to use a tiny $\eta = 0.01$. From $(p_0, p_1) = (1.5, 10)$:

k	p_0	p_1	$g_0 = 100p_0$	$g_1 = 2p_1$	Δ_0	Δ_1
0	1.500	10.000	150.0	20.0	-1.500	-0.200
1	0.000	9.800	0.0	19.6	0.000	-0.196
2	0.000	9.604	0.0	19.2	0.000	-0.192
3	0.000	9.412	0.0	18.8	0.000	-0.188
4	0.000	9.224	—	—	—	—

The result. p_0 converged in *one step*. Perfect for p_0 . But p_1 is still at 9.22 after 4 steps, barely changed. *The same η that's right for p_0 is cripplingly small for p_1 .*

Diagnosis. A single global η cannot be right for parameters with very different sensitivities. We need a *per-parameter* learning rate.

Solution 2: AdaGrad gives each parameter its own rate

The fix. Maintain a per-parameter accumulator of squared gradients, and divide η by its square root:

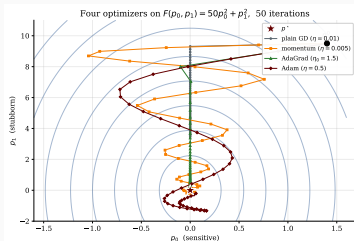
$$G_{k,i} = G_{k-1,i} + g_{k,i}^2, \quad p_{k+1,i} = p_{k,i} - \frac{\eta_0}{\sqrt{G_{k,i}}} g_{k,i}.$$

Let $\eta_0 = 1.5$ (aggressive). Effective per-parameter rate: $\eta_{\text{eff},i} = \eta_0 / \sqrt{G_{k,i}}$.

Iteration 0 on the same problem.

Step	For p_0	For p_1
gradient g	$g_0 = 100 \cdot 1.5 = 150$	$g_1 = 2 \cdot 10 = 20$
update accumulator	$G_0 = 0 + 150^2 = 22,500$	$G_1 = 0 + 20^2 = 400$
effective rate	$\eta_0 / \sqrt{22500} = 1.5/150 = 0.01$	$\eta_0 / \sqrt{400} = 1.5/20 = 0.075$
update	$\Delta_0 = -0.01 \cdot 150 = -1.5$	$\Delta_1 = -0.075 \cdot 20 = -1.5$
new value	$p_0 = 1.5 - 1.5 = 0.0$	$p_1 = 10 - 1.5 = 8.5$

Both parameters got a step of size 1.5. AdaGrad automatically discovered $\eta_0 = 0.01$ for the sensitive coordinate and $\eta_1 = 0.075$ for the stubborn one (7.5× bigger). **No hand-tuning.**



All four optimizers, same starting point, 50 iterations. Plain GD nails p_0 in one step but crawls down p_1 . Momentum, AdaGrad, and Adam all converge cleanly — but for very different reasons (momentum: velocity accumulation; AdaGrad: per-coordinate η ; Adam: both).

AdaGrad's rate dies → Solution: forgetful averages

The flaw. The accumulator $G_k = \sum_{j \leq k} g_j^2$ **only ever increases**. It never goes down.

Long-run consequence. After many iterations, G_k is huge, so $\eta_0/\sqrt{G_k}$ is tiny, so the effective step size is tiny. *Learning grinds to zero, no matter how far we still are from the minimum.*

This is the **dying learning rate problem**.

The diagnosis. We want adaptive per-parameter rates — but we don't want them to grow without bound. We need to *forget* the distant past.

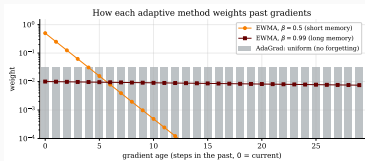
Solution 3: the forgetful average (EWMA). Replace the cumulative sum with an exponentially weighted moving average:

$$v_k = \beta v_{k-1} + (1 - \beta) g_k^2.$$

$\beta \in (0, 1)$ is the *memory factor*.

β value	Behavior
β high (e.g., 0.99)	long memory; slow adaptation; gentle averaging.
β low (e.g., 0.1)	short memory; fast adaptation; reacts quickly to new gradients.

Unlike AdaGrad's cumulative sum, v_k *forgets* old gradients exponentially. Old contributions decay as β^{age} , so the effective rate doesn't shrink to zero. This recipe is **RMSProp**.



Gray bars: AdaGrad weights all past gradients equally (heights normalized to sum to 1). Orange: EWMA with $\beta = 0.5$ — recent gradients dominate. Maroon: EWMA with $\beta = 0.99$ — gentler decay, longer memory but still finite.

What's still missing. RMSProp adapts per-coordinate *magnitude* but discards momentum (smoothed direction). Adam combines both.

The synthesis: Adam

Combine all three ideas. Adam runs two forgetful averages:

$$\begin{aligned}\mathbf{m}_k &= \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \mathbf{g}_k && \text{(EWMA of } \mathbf{g} \text{ — direction, the momentum part)} \\ \mathbf{v}_k &= \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) \mathbf{g}_k^2 && \text{(EWMA of } \mathbf{g}^2 \text{ — magnitude, the RMSProp part)}\end{aligned}$$

Defaults: $\beta_1 = 0.9$ (momentum, longer memory), $\beta_2 = 0.999$ (magnitude, very long memory).

One more issue: the cold start. $\mathbf{m}_0 = \mathbf{v}_0 = 0$ biases the early estimates toward zero. **Bias correction:**

$$\hat{\mathbf{m}}_k = \mathbf{m}_k / (1 - \beta_1^k), \quad \hat{\mathbf{v}}_k = \mathbf{v}_k / (1 - \beta_2^k).$$

The factor $1 - \beta^k$ is small when k is small (boosts the early estimate) and approaches 1 as k grows (correction fades).

The Adam update.

$$\theta_{k+1} = \theta_k - \eta \frac{\hat{\mathbf{m}}_k}{\sqrt{\hat{\mathbf{v}}_k} + \varepsilon}.$$

The smooth direction of momentum, scaled per-coordinate by the inverse of an EWMA of squared gradients. *Three ideas, one update line.*

The story in one breath. GD has no memory $\Rightarrow$ momentum. Momentum's η is global $\Rightarrow$ AdaGrad. AdaGrad's G grows forever $\Rightarrow$ EWMA $\Rightarrow$ RMSProp. EWMAs start biased $\Rightarrow$ bias correction. **Adam = momentum (EWMA of $\mathbf{g}$) + RMSProp (EWMA of $\mathbf{g}^2$) + bias correction.**

Adam, traced through one iteration: $f(p) = p^2$ from $p_0 = 10$

Setup. $p_0 = 10$, $\mathbf{m}_0 = 0$, $\mathbf{v}_0 = 0$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 1.05$, $\epsilon \approx 0$.

($\eta = 1.05$ is chosen so the arithmetic comes out clean; in real DL training $\eta \sim 10^{-3}$.)

Step at $k = 1$	Formula	Substitute	Value
1. compute gradient	$g_1 = f'(p_0) = 2p_0$	$2 \cdot 10$	20
2. update $\mathbf{m}$	$m_1 = \beta_1 m_0 + (1 - \beta_1)g_1$	$0.9 \cdot 0 + 0.1 \cdot 20$	2.0
3. update $\mathbf{v}$	$v_1 = \beta_2 v_0 + (1 - \beta_2)g_1^2$	$0.999 \cdot 0 + 0.001 \cdot 400$	0.4
4. bias-correct $\mathbf{m}$	$\hat{m}_1 = m_1 / (1 - \beta_1^1)$	$2.0 / 0.1$	20.0
5. bias-correct $\mathbf{v}$	$\hat{v}_1 = v_1 / (1 - \beta_2^1)$	$0.4 / 0.001$	400.0
6. compute the step	$\Delta = -\eta \hat{m}_1 / \sqrt{\hat{v}_1}$	$-1.05 \cdot 20 / \sqrt{400}$	-1.05
7. new parameter	$p_1 = p_0 + \Delta$	$10 - 1.05$	8.95

Why bias correction is doing real work here. Without it, $m_1 / \sqrt{v_1} = 2 / \sqrt{0.4} \approx 3.16$, scaled by η gives a step of ~ 3.3 . *Wrong scale.* With bias correction, $\hat{m}_1 / \sqrt{\hat{v}_1} = 20 / 20 = 1$ exactly — the step size matches η . *The two corrections cancel in opposite directions and recover a properly scaled step.*

Continuing for 10 iterations (same setup): p_k traces

$10.0 \rightarrow 8.95 \rightarrow 7.90 \rightarrow 6.87 \rightarrow 5.84 \rightarrow 4.84 \rightarrow 3.86 \rightarrow 2.92 \rightarrow 2.02 \rightarrow 1.17 \rightarrow \mathbf{0.38}$.

Smooth, monotone descent — no oscillation, no premature stall. *Compare:* plain GD with $\eta = 0.1$ gives $p_{10} \approx 1.07$. **Adam is $\sim 3\times$ closer with no hand-tuning.**

Adam, in four problems and four fixes

The story of how Adam was built. Each step solves a specific failure of the previous. Walk it as an *engineering arc*, not a list of algorithms.

Problem	Fix
1. GD is memoryless. On $f(p) = p^2$ from $p_0 = 10$ with $\eta = 0.1$: step shrinks with the gradient. After 5 steps, $p \approx 3.28$ — a crawl.	Momentum. $\mathbf{v}_k = \beta \mathbf{v}_{k-1} + \mathbf{g}_k$, then $\theta \leftarrow \theta - \eta \mathbf{v}_k$. Velocity carries over between steps. With $\beta = 0.5$ on the same problem: after 5 steps, $p \approx 0.084$ — $40\times$ closer.
2. One global η is wrong. Different parameters have wildly different sensitivities. On $F(p_0, p_1) = 50 p_0^2 + p_1^2$: a small η that keeps p_0 from blowing up makes p_1 crawl.	Per-coordinate rate — AdaGrad. $\eta_i = \eta_0 / \sqrt{\sum_{j \leq k} g_{j,i}^2}$. Gives each parameter its own η from its own gradient history. Sensitive coords get tiny η , stubborn ones get big.
3. AdaGrad's accumulator grows forever. The denominator never decreases, so the effective rate dies.	Forgetful average — EWMA. $\mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) \mathbf{g}_k^2$. The recent past dominates; the distant past is forgotten. (RMSProp.)
4. Cold start. EMAs initialized at 0 underestimate true magnitudes for the first few iterations.	Bias correction. Divide by $1 - \beta^k$. Boosts early estimates; correction fades to 1 as k grows.

Take-away

Adam = momentum + RMSProp + bias correction. *Each piece is the answer to a specific failure of the previous step on this list.*

AdaGrad in detail

Algorithm [Duchi–Hazan–Singer 2011, JMLR 12]. For coordinate i :

$$G_{k,i} = G_{k-1,i} + \mathbf{g}_{k,i}^2, \quad \theta_i^{k+1} = \theta_i^k - \frac{\eta}{\sqrt{G_{k,i} + \varepsilon}} \mathbf{g}_{k,i}.$$

Stores one scalar per parameter. The cumulative sum $G_{k,i}$ is monotonically increasing, so the effective step size in each coordinate is monotonically *decreasing*.

Theorem (Duchi et al. 2011, Theorem 5)

For convex F , AdaGrad with appropriate η achieves regret

$$R_K = \sum_{k=1}^K [F(\theta^k) - F(\theta^*)] \leq \sqrt{2} D_\infty \sum_{i=1}^p \sqrt{\sum_{k=1}^K \mathbf{g}_{k,i}^2},$$

where D_∞ bounds $\|\theta^k - \theta^*\|_\infty$. Per-coordinate bound: each coordinate's regret depends on its own gradient history.

Why this is striking. On problems with sparse gradients — e.g., text classification with one-hot features — most coordinates have $\mathbf{g}_{k,i} = 0$ most of the time. AdaGrad's rate then depends on the gradient magnitude only on the active coordinates, and the inactive ones contribute zero. Plain SGD with a single global step size can't exploit this.

The flaw. $G_{k,i}$ grows without bound, so step sizes shrink to zero too aggressively in long-running training. Once you're past the regime where the cumulative sum reflects the current curvature, AdaGrad just stops learning. **This is exactly what RMSProp fixes.**

RMSProp and Adam

RMSProp [Hinton, lecture notes 2012; Tieleman–Hinton 2012, Coursera]. Replace AdaGrad's cumulative sum with an exponential moving average:

$$\mathbf{v}_i^{k+1} = \beta_2 \mathbf{v}_i^k + (1 - \beta_2) \mathbf{g}_{k,i}^2, \quad \theta_i^{k+1} = \theta_i^k - \frac{\eta}{\sqrt{\mathbf{v}_i^{k+1} + \varepsilon}} \mathbf{g}_{k,i}.$$

Default $\beta_2 = 0.9$. The EMA *forgets* old gradient magnitudes, so the effective step doesn't shrink to zero.

Adam [Kingma–Ba 2015, ICLR; Bishop & Bishop 2024, Algorithm 7.4]. RMSProp + momentum + bias correction. *Each line below is highlighted by which problem it solves:*

$\mathbf{m}^{k+1} = \beta_1 \mathbf{m}^k + (1 - \beta_1) \mathbf{g}_k$	← momentum (EMA of $\mathbf{g}$)
$\mathbf{v}^{k+1} = \beta_2 \mathbf{v}^k + (1 - \beta_2) \mathbf{g}_k^2$	← RMSProp (EMA of $\mathbf{g}^2$)
$\hat{\mathbf{m}}^{k+1} = \mathbf{m}^{k+1} / (1 - \beta_1^{k+1}), \quad \hat{\mathbf{v}}^{k+1} = \mathbf{v}^{k+1} / (1 - \beta_2^{k+1})$	← bias correction
$\theta^{k+1} = \theta^k - \eta \frac{\hat{\mathbf{m}}^{k+1}}{\sqrt{\hat{\mathbf{v}}^{k+1} + \varepsilon}}$	← the update

$\mathbf{g}_k^2$ and division are elementwise. Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$, $\varepsilon = 10^{-8}$.

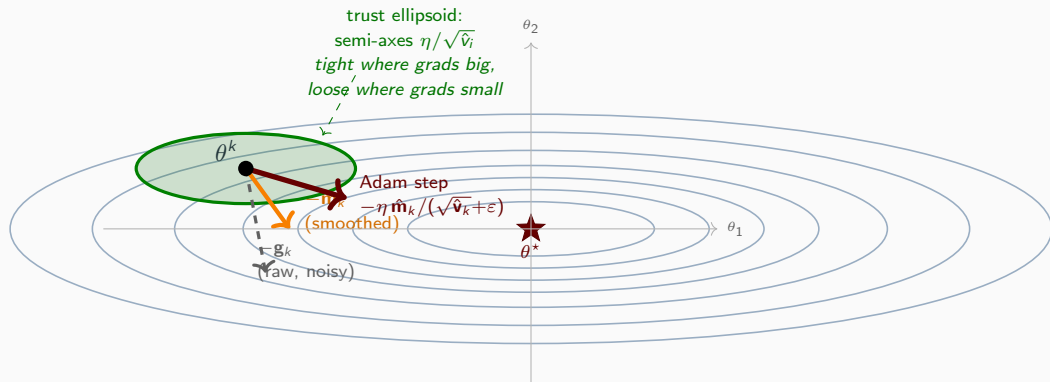
Reading the algorithm.

- **Orange ($\mathbf{m}$):** the *first-moment* EMA — a smoothed gradient direction.
- **Blue ($\mathbf{v}$):** the *second-moment* EMA — a per-coordinate magnitude estimate.
- **Green ($\hat{\mathbf{m}}, \hat{\mathbf{v}}$):** bias correction matters only in the first ~ 100 steps ; afterwards $1 - \beta^k \approx 1$.
- **Maroon (update):** when the gradient is consistent, the effective step is $\approx \eta \text{sign}(\mathbf{g}_i)$ per coordinate.

Take-away

Adam = momentum + RMSProp + bias correction, all in five lines.
The default optimizer behind nearly every modern transformer training run.

Anatomy of one Adam step: all the vectors



$-g_k$ (gray dashed) = raw stochastic gradient | $-\hat{m}_k$ (orange) = bias-corrected first-moment EMA | green ellipse = trust ellipsoid | Adam step (maroon) = $-\eta \hat{m}_k / (\sqrt{\hat{v}_k} + \epsilon)$.

Reading the picture. The raw gradient (dashed) points strongly across the narrow valley — typical for plain GD. After EMA smoothing, $-\hat{m}_k$ is in a similar direction but cleaner. **Adam's per-coordinate division by $\sqrt{\hat{v}}$ rotates the step toward the long axis** (because $\hat{v}_2 \gg \hat{v}_1$ on this valley), landing inside the trust ellipsoid and pointed at θ^* .

Adam as a trust-region method

Classical trust-region method [Powell 1970; Conn–Gould–Toint 2000].

$$\delta^* = \arg \min_{\|\delta\| \leq \Delta_k} \underbrace{F(\theta^k) + \nabla F(\theta^k)^\top \delta + \frac{1}{2} \delta^\top H_k \delta}_{\text{local quadratic model}}.$$

Adapt Δ_k based on whether the model predicted the actual decrease.

Adam is this template with three specific choices:

- *Gradient*: smoothed $\hat{\mathbf{m}}_k$, not raw ∇F .
- *Hessian*: $H_k = \frac{1}{\eta} \text{diag}(\sqrt{\hat{\mathbf{v}}_k})$ — diagonal, anisotropic, gradient-magnitude-based.
- *Region*: ellipsoid with semi-axes $\propto 1/\sqrt{\hat{v}_{k,i}}$. *The green ellipse on the previous slide.*

The Adam subproblem separates per-coordinate; closed-form solution is exactly Adam:

$$\delta_i^* = -\eta \hat{m}_{k,i} / \sqrt{\hat{v}_{k,i}}.$$

Differences from classical TR. Adam keeps η *fixed* but adapts the region's *shape*; uses gradient magnitudes (not real Hessian); diagonal closed-form (no CG subsolve). *Sophia* replaces $\sqrt{\hat{\mathbf{v}}}$ with a real diagonal-Hessian estimate — “Adam done with real curvature.”

The framing. Every modern adaptive optimizer = a choice of (i) gradient estimate, (ii) Hessian estimate, (iii) radius rule.

Why bias correction matters

The bias. If $\mathbf{m}^0 = \mathbf{0}$ and $\mathbf{m}^{k+1} = \beta_1 \mathbf{m}^k + (1 - \beta_1) \mathbf{g}_k$ with $\mathbf{g}_k$ i.i.d. from a stationary distribution with mean $\mathbf{g}^*$,

$$\mathbb{E}[\mathbf{m}^{k+1}] = (1 - \beta_1^{k+1}) \mathbf{g}^*.$$

Proof. Unroll: $\mathbf{m}^{k+1} = (1 - \beta_1) \sum_{j=0}^k \beta_1^{k-j} \mathbf{g}_j$. Take expectation:

$$(1 - \beta_1) \sum_{j=0}^k \beta_1^{k-j} \mathbf{g}^* = (1 - \beta_1) \mathbf{g}^* \frac{1 - \beta_1^{k+1}}{1 - \beta_1} = (1 - \beta_1^{k+1}) \mathbf{g}^*. \quad \square$$

So $\mathbf{m}^{k+1}$ is a biased estimate of $\mathbf{g}^*$ when k is small. With $\beta_1 = 0.9$, after one update step ($k = 0$, so the exponent is $k + 1 = 1$): bias factor is $1 - 0.9^1 = 0.1$ — the EMA underestimates the gradient by $10\times$. After two steps: $1 - 0.9^2 = 0.19$ ($\sim 5\times$ underestimate). After $k + 1 = 50$: factor ≈ 1 , bias is gone.

The correction. Divide by $1 - \beta_1^{k+1}$:

$$\hat{\mathbf{m}}^{k+1} = \mathbf{m}^{k+1} / (1 - \beta_1^{k+1}) \Rightarrow \mathbb{E}[\hat{\mathbf{m}}^{k+1}] = \mathbf{g}^*.$$

Same idea for $\mathbf{v}$ with β_2 .

Why it's not just cosmetic. Without bias correction, the effective step size in iteration 1 is way too small (because $\mathbf{m}^1 \approx 0.1 \mathbf{g}_0$), and the early training dynamics are wrong.

Bias-corrected Adam takes a properly-sized first step. *This is the only piece of Adam that's not in plain RMSProp + momentum.*

Tradition of confusion. Many practitioners assume the bias correction is responsible for Adam's advantage over RMSProp + momentum. It is not — the correction matters mostly in the first ~ 100 steps. Adam's empirical advantage over RMSProp + momentum in long training runs is small or zero; what matters is the per-coordinate scaling and the bias correction's effect on the warm-up phase.

AdamW and Lion: the modern descendants

The bug AdamW fixes. Naively adding $\lambda\theta$ to the gradient inside Adam means the regularizer gets scaled by $1/\sqrt{\hat{\mathbf{v}}}$ per coordinate — effective regularization strength varies coordinate-by-coordinate. *Not what you want.*

AdamW [Loshchilov & Hutter 2019]. Apply weight decay *outside* the adaptive scaling:

$$\theta^{k+1} = \theta^k - \eta \frac{\hat{\mathbf{m}}^{k+1}}{\sqrt{\hat{\mathbf{v}}^{k+1}} + \epsilon} - \eta \lambda \theta^k.$$

On transformers (BERT, GPT, ViT), beats Adam-with- ℓ_2 by 1–3% test accuracy. **The default in 2025.**

Lion [Xiangning Chen, C. Liang, D. Huang, E. Real, Q. Le et al. 2023, NeurIPS, “Symbolic Discovery of Optimization Algorithms”]. Sign-based; half the state of Adam:

$$\theta^{k+1} = \theta^k - \eta \text{sign}(\beta_2 \mathbf{m}^k + (1 - \beta_2) \mathbf{g}_k) - \eta \lambda \theta^k.$$

Discovered by symbolic regression over algorithmic primitives. *The optimizer wasn’t designed; it was found.*

Roadmap

From Yesterday's Theory to Today's Practice

The DL Training Problem

ADMM is not Adam

Per-coordinate Adaptive Methods

When Adam Fails, and Why

Second-Order Revival

Loss-Landscape Phenomena

Wrap-Up

Adam can fail to converge — the Reddi–Kale–Kumar counterexample

Theorem (Reddi, Kale, Kumar 2018, ICLR best paper, “On the Convergence of Adam and Beyond”)

There exists a stochastic 1-D online convex problem on which Adam (with default $\beta_1 = 0.9$, $\beta_2 = 0.999$) fails to converge to the optimum — the iterates drift away from x^ .*

The mechanism. A stochastic gradient with rare large value C (prob p) and frequent small value -1 (prob $1 - p$). The rare C 's dominate $\mathbf{v}$, shrinking the positive-direction step; Adam drifts the wrong way.

Fix — AMSGrad. Replace the $\mathbf{v}$ EMA with the running maximum:

$$\mathbf{v}^{k+1} = \max(\mathbf{v}^k, \beta_2 \mathbf{v}^k + (1 - \beta_2) \mathbf{g}_k^2).$$

Effective step never increases; convergence is recovered. In practice AMSGrad and Adam perform similarly on real problems — the failure mode rarely triggers.

Theoretical state of Adam in 2025. Convergence of Adam under realistic assumptions is still an open problem. There are positive results under exotic conditions (Zhang et al. 2022; Défossez et al. 2022), but no clean theorem that matches what practitioners observe.

Roadmap

From Yesterday's Theory to Today's Practice

The DL Training Problem

ADMM is not Adam

Per-coordinate Adaptive Methods

When Adam Fails, and Why

Second-Order Revival

Loss-Landscape Phenomena

Wrap-Up

Why second-order methods are coming back

Classical wisdom (2010s). Second-order methods (Newton, quasi-Newton) were considered impractical for deep learning: $H \in \mathbb{R}^{p \times p}$ with $p \sim 10^9$ is uncomputable; even storing H would take $\sim p^2/8$ exabytes.

What changed.

1. *Structure-aware approximations.* H has block-diagonal / Kronecker-product structure when θ is the parameters of a layered network. Exploit it.
2. *Hessian-vector products.* Hv can be computed in one forward + backward pass — $O(p)$ time, not $O(p^2)$ — using auto-diff. So we never need to materialize H to use it.
3. *The Adam plateau.* For very large models, AdamW seems to hit performance ceilings that more curvature-aware methods can break.

Three modern second-order methods.

Method	Curvature approximation	Year	Use
K-FAC	Kronecker-factored Fisher info per layer	2015 (Martens–Grosse)	Image classification, RL
Shampoo	Tensor-product preconditioning per layer	2018 (Gupta–Koren–Singer)	Deep nets, recommender systems
Sophia	Diagonal-Hessian estimate, clipped	2023 (Liu, Ren, Tian, Liang, Tang)	Foundation-model pretraining

The common theme. Estimate something Hessian-shaped using the structure of layered networks; precondition the gradient by its inverse; take a step. None of these is a full Newton method — they're all approximate-curvature first-order methods.

Two prerequisites: Fisher information and the Kronecker product

Fisher information matrix. For a parametric distribution $p_\theta(x)$, the score is the gradient of the log-likelihood, and the *Fisher information* is its covariance:

$$F(\theta) := \mathbb{E}_{x \sim p_\theta} [\nabla_\theta \log p_\theta(x) \nabla_\theta \log p_\theta(x)^\top].$$

Equivalently (under regularity), F is the negative expected Hessian of the log-likelihood:

$$F(\theta) = -\mathbb{E}_{x \sim p_\theta} [\nabla_\theta^2 \log p_\theta(x)].$$

For a neural net. The output distribution $p_\theta(y | \mathbf{x})$ defines a Fisher matrix on the parameter space. When the model fits the training data well, $F(\theta)$ is approximately the Hessian of the cross-entropy loss.

Three reasons to precondition by F^{-1} instead of H^{-1} .

- $F \succeq 0$ always (it's a covariance). H can be indefinite at saddles.
- F uses only gradients — no second derivatives.
- *Natural gradient* [Amari 1998]: $-F^{-1} \nabla F$ is the steepest-descent direction in the KL-divergence geometry on the space of distributions p_θ , not the Euclidean geometry on parameters.
Reparameterization-invariant.

The connection (next slide). Inside a neural net, the Fisher block of layer ℓ is approximately a Kronecker product $A_\ell \otimes G_\ell$, where A_ℓ is the input-activation covariance and G_ℓ is the gradient covariance. K-FAC exploits exactly this structure to precondition with the Fisher information at the cost of two small inversions per layer.

Kronecker product. For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times q}$,

$$A \otimes B := \begin{pmatrix} a_{11}B & a_{12}B & \cdots & a_{1n}B \\ a_{21}B & a_{22}B & \cdots & a_{2n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mn}B \end{pmatrix} \in \mathbb{R}^{mp \times nq}.$$

Every entry of A scales a copy of B , tiled into a big matrix.

Three identities (this is what K-FAC exploits).

- $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$.
- $\boxed{(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}}$ (when both inverses exist).
- $\text{vec}(B X A^\top) = (A \otimes B) \text{vec}(X)$ (the “vec trick”).

Computational savings. Suppose we want $(A \otimes B)^{-1} \mathbf{v}$ for $\mathbf{v} \in \mathbb{R}^{mp}$.

- *Naïve:* form the $mp \times mp$ matrix, invert it. Cost $O((mp)^3)$. Storage $(mp)^2$ entries.
- *Kronecker:* invert A ($O(m^3)$) and B ($O(p^3)$); apply via the vec trick. Cost $O(m^3 + p^3 + m^2p + mp^2)$. Storage $m^2 + p^2$ entries.

For $m = p = 1000$: 10^9 vs 10^{18} flops. The difference between possible and impossible.

K-FAC: Kronecker-factored approximate curvature

Setup. A neural network has many *layers*. The parameters of layer ℓ form a matrix $W_\ell \in \mathbb{R}^{m \times n}$. The Fisher information matrix block for layer ℓ is approximately

$$F_\ell \approx A_\ell \otimes G_\ell, \quad A_\ell \in \mathbb{R}^{m \times m}, \quad G_\ell \in \mathbb{R}^{n \times n},$$

where A_ℓ is the covariance of the layer's input activations and G_ℓ is the covariance of the gradient with respect to the pre-activation outputs. *Kronecker structure* — much smaller than the full $mn \times mn$ Fisher block.

Update [Martens & Grosse 2015, ICML, “Optimizing Neural Networks with Kronecker-factored Approximate Curvature”]:

$$W_\ell^{k+1} = W_\ell^k - \eta (A_\ell + \rho I)^{-1} \nabla_{W_\ell} F (G_\ell + \rho I)^{-1}.$$

The Kronecker identity $(A \otimes G)^{-1} = A^{-1} \otimes G^{-1}$ means we never invert $A_\ell \otimes G_\ell$ directly — two small inversions instead of one big one.

Cost per step. $O(m^3 + n^3)$ per layer, vs. $O((mn)^3)$ for full Newton on the layer. For a layer with $m = n = 1000$ this is 10^9 vs 10^{18} — the difference between possible and impossible.

Practical use. K-FAC was the first second-order method to convincingly beat first-order methods on convolutional nets. It's used internally at DeepMind (e.g., for AlphaZero training). The catch: the Kronecker structure assumption breaks down for some architectures (attention, normalization layers), and practical implementations are intricate.

Reading. Martens, “New insights and perspectives on the natural gradient method” (JMLR 2020) is the comprehensive treatment.

Shampoo and Sophia

Shampoo [Gupta–Koren–Singer 2018, ICML]. Generalize K-FAC’s Kronecker structure to higher-order tensors. For a parameter *tensor* $W \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$, maintain d preconditioner matrices $P_1, P_2, \dots, P_d$ where P_i is the second-moment statistic along the i -th tensor axis. Update:

$$W^{k+1} = W^k - \eta P_1^{-1/4} \dots P_d^{-1/4} \mathbf{g}_k.$$

Each P_i has size $n_i \times n_i$. Total state $O(\sum_i n_i^2)$ vs. AdaGrad’s $O(\prod_i n_i)$. *Tensor preconditioning at a fraction of the cost.*

Used in Google’s recommender-system training. Recently shown to outperform AdamW in foundation-model pretraining when scaled properly [Anil et al. 2024].

Sophia [Liu, Ren, Tian, Liang, Tang 2023, “Sophia: A Scalable Stochastic Second-order Optimizer”]. *Diagonal-Hessian* estimation via Hessian-vector products + per-coordinate clipping. With momentum $\mathbf{m}_k$ on the gradient and an EMA $\mathbf{h}_k$ of the diagonal Hessian estimate:

$$\mathbf{h}_k \approx \text{diag}(H(\theta^k)) \quad (\text{stochastic estimate via } H\nu), \quad \theta^{k+1} = \theta^k - \eta \text{clip}\left(\frac{\mathbf{m}_k}{\max(\gamma \mathbf{h}_k, \varepsilon)}, \rho\right).$$

ρ is the clip threshold (typically 1); γ scales the Hessian estimate; ε is a small positive floor. Clipping prevents update blow-up where $\mathbf{h}_k \approx 0$ (small-curvature directions). Storage is $O(p)$ — same as Adam.

Result. Sophia trains GPT-2-style models in $\sim 50\%$ fewer iterations than AdamW. For pretraining a 770M-parameter model, this is the difference between \$10K and \$20K of compute — so the algorithm is in production at several foundation-model labs.

The trend. Production-scale deep learning is increasingly willing to pay $\sim 30\%$ extra per-iteration cost (one Hessian-vector product + book-keeping) to halve the iteration count. This is the inverse of the 2015 deep-learning conventional wisdom.

Roadmap

From Yesterday's Theory to Today's Practice

The DL Training Problem

ADMM is not Adam

Per-coordinate Adaptive Methods

When Adam Fails, and Why

Second-Order Revival

Loss-Landscape Phenomena

Wrap-Up

The “edge of stability” phenomenon

Classical descent-lemma theory says. For L -smooth f and step size $\eta = 1/L$, GD's per-step decrease is $f(\mathbf{x}^{k+1}) \leq f(\mathbf{x}^k) - \frac{1}{2L} \|\nabla f\|^2$. Bigger η than $2/L$, and the algorithm can diverge.

Cohen, Kaur, Li, Kolter, Talwalkar 2021 (ICLR 2022, “Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability”) observed something completely different in practice on neural-network training:

1. Pick a learning rate η .
2. Train for a few hundred steps.
3. Measure $\lambda_{\max}(H(\theta^k))$ (the local Lipschitz constant) along the trajectory.
4. It rises until $\lambda_{\max} \approx 2/\eta$, then *stabilizes there*.

This is impossible under classical theory. If $\lambda_{\max} > 2/\eta$, GD should diverge. It doesn't.

What's actually happening. The trajectory enters a regime where the descent lemma is violated on individual steps but the loss still decreases on *average over consecutive steps*. The iterate oscillates across a sharp valley, but each oscillation is followed by progress along the valley's long axis. The classical analysis gives a too-conservative bound; the actual non-monotone progress is real.

Implications.

- GD on neural nets is *not* in the regime where descent-lemma proofs apply.
- Sharpness $\lambda_{\max}$ is a function of η , not of the problem alone.
- Larger $\eta \Rightarrow$ flatter solution (smaller $\lambda_{\max}$). **Implicit regularization** in the optimizer.
- “Edge of stability” is now considered the *default* regime for neural-network training, not an exception.

Reading. Cohen et al. 2021. Also Damian, Nichani & Lee (2022) for a theoretical explanation in the simplest setting.

Critical batch size and large-batch training

Recall (from yesterday's mini-batch slide): mini-batch SGD's variance scales as σ^2/b , so noise-floor terms in convergence bounds scale as $1/b$. If you can afford b -times-larger batches, you can run b -times-fewer iterations.

The catch: the speedup saturates beyond a problem-dependent **critical batch size** B^* .

Theorem (McCandlish, Kaplan, Amodei et al. 2018, "An Empirical Model of Large-Batch Training")

For SGD on a smooth F , the speedup over single-example SGD as a function of batch size b is approximately

$$\text{speedup}(b) \approx \frac{b}{1 + b/B^*}, \quad B^* = \frac{\text{tr}(H\Sigma)}{\mathbf{g}^\top H \mathbf{g}} \approx \frac{\sigma^2}{\|\nabla F\|^2} \text{ (simple-noise approximation).}$$

Here Σ is the per-example gradient covariance and $\mathbf{g} = \nabla F$. For $b \ll B^$: linear speedup. For $b \gg B^*$: no speedup — you're just spending more compute per step for the same total progress.*

Practical numbers. For ImageNet training, $B^* \approx 8K$. For GPT-3-scale language pretraining, $B^* \approx 3.2M$ tokens per batch. Beyond B^* :

- Linear scaling rule (double $b \Rightarrow$ double η) breaks.
- Need warmup, layer-wise adaptive rates (LARS, LAMB), or per-coordinate methods (Adam) to keep training stable.

Goyal, Dollár, Girshick, Noordhuis et al. 2017 ("Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour"): linear scaling rule + warmup gets ResNet-50 to 76% top-1 in 1 hour on a 256-GPU cluster, training with $b = 8192$. *Practical recipe for large-batch training.*

LAMB [You, Li, Reddi et al. 2020, ICLR]: per-layer adaptive scaling. The optimizer behind training BERT in 76 minutes on a TPU pod.

Flat vs. sharp minima, and Sharpness-Aware Minimization (SAM)

The conjecture. Hochreiter & Schmidhuber (1997) “Flat Minima”: among the many global minima of a deep network’s loss, the *flat* ones — where the Hessian has small eigenvalues — generalize better than *sharp* ones. Intuition: a flat minimum corresponds to a robust solution; a sharp minimum is an over-fit.

Empirical confirmation [Keskar, Mudigere, Nocedal, Smelyanskiy, Tang 2017, ICLR, “On Large-Batch Training for Deep Learning”]. Large-batch SGD finds sharper minima than small-batch SGD; the test accuracy gap correlates with the sharpness gap.

Critique [Dinh, Pascanu, Bengio, Bengio 2017, ICML, “Sharp Minima Can Generalize for Deep Nets”]. Sharpness is *not* reparameterization-invariant: rescaling $W_\ell \rightarrow W_\ell/\alpha$ and $W_{\ell+1} \rightarrow \alpha W_{\ell+1}$ leaves the function unchanged but multiplies the Hessian by α^2 . So “sharpness” as the Hessian norm is meaningful only relative to a fixed parameterization.

Sharpness-Aware Minimization (SAM) [Foret, Kleiner, Mobahi, Neyshabur 2021, ICLR, “Sharpness-Aware Minimization for Efficiently Improving Generalization”]. Replace the standard ERM problem with a minimax that explicitly penalizes sharpness:

$$\min_{\theta} \max_{\|\delta\| \leq \rho} F(\theta + \delta).$$

The inner maximization is over a small ball of radius ρ around θ — the worst-case loss in a neighborhood. **Solutions of this minimax are necessarily flat.**

Practical algorithm. The inner max is approximated by one gradient step:

$$\delta^*(\theta) \approx \rho \frac{\nabla F(\theta)}{\|\nabla F(\theta)\|}.$$

Then take the outer SGD step at the perturbed point:

$$\theta^{k+1} = \theta^k - \eta \nabla F(\theta^k + \delta^*(\theta^k)).$$

Cost: 2 gradient evaluations per step instead of 1. A $2\times$ wall-clock penalty for a 0.5–2% test-accuracy gain.

Used in production. ViT pretraining, ResNet variants, NLP fine-tuning. Also has principled extensions: ASAM (adaptive SAM, Kwon et al. 2021), GSAM (Zhuang et al. 2022). The minimax-with-inner-perturbation pattern is one of the few rigorous optimization-side ideas that improves generalization out-of-the-box on big models.

Reading. Bahri, Foret & Cubuk (2022), *Sharpness-Aware Minimization Improves Language Model Generalization*, ACL. The SAM line of work is one of the cleanest pieces of optimization theory $\rightarrow$ DL practice in the last five years.

Implicit bias of (S)GD: optimizer choice picks a solution

The puzzle. On an over-parameterized neural network, the loss surface has *infinitely many global minima* — whole connected manifolds of θ 's with $F(\theta) = 0$. Two networks with identical training loss can generalize very differently. *Which minimum does SGD pick, and why?*

The cleanest theorem [Soudry, Hoffer, Nacson, Gunasekar, Srebro 2018, JMLR 19, "The Implicit Bias of Gradient Descent on Separable Data"]. Even on the simplest over-parameterized problem — linear logistic regression on linearly-separable data — GD with no regularizer converges to a specific direction:

Theorem

Let $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ be linearly separable, $y_i \in \{-1, +1\}$. Run GD on the logistic loss $F(\theta) = \frac{1}{n} \sum_i \log(1 + e^{-y_i \theta^\top \mathbf{x}_i})$ with constant step $\eta < 1/L$. Then $\|\theta^k\| \rightarrow \infty$, but the direction converges to the max-margin solution:

$$\frac{\theta^k}{\|\theta^k\|} \rightarrow \frac{\theta_{\text{SVM}}^*}{\|\theta_{\text{SVM}}^*\|}, \quad \text{where } \theta_{\text{SVM}}^* = \arg \min_{\frac{1}{2} \|\theta\|^2 \text{ s.t. } y_i \theta^\top \mathbf{x}_i \geq 1 \forall i}.$$

The same direction the hard-margin SVM picks. The convergence rate is $O(1/\log k)$ — glacial, but the direction is correct.

Why this happens. The logistic loss is $\ell(z) \approx e^{-z}$ as $z \rightarrow \infty$. To drive $F \rightarrow 0$ on separable data, θ must grow in the direction that maximizes $\min_i y_i \theta^\top \mathbf{x}_i$ — exactly the SVM max-margin direction. *No margin term appears in the loss; the optimizer chooses one anyway.*

What this says about Adam. Wilson, Roelofs, Stern, Srebro & Recht (2017, NeurIPS, "The Marginal Value of Adaptive Gradient Methods in Machine Learning") showed that on the same problem, *Adam converges to a different direction* — one with worse generalization than SGD's max-margin direction. **Optimizer choice is a regularizer.**

Beyond linear models. For deep networks the implicit-bias picture is messier — different parameterizations give different implicit norms (Gunasekar et al. 2018), and the bias depends on initialization. But the principle holds: optimization and generalization are coupled in over-parameterized models. *Which minimum you find is part of what defines the algorithm, not just the loss.*

Connections. Lyu & Li (2020), Chizat & Bach (2020), Ji & Telgarsky (2020). All extend Soudry et al. to deep nets and other losses.

Lottery tickets and implicit bias

Two more loss-landscape phenomena worth knowing.

Lottery Ticket Hypothesis [Frankle & Carbin 2019, ICLR best paper, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks”]. Inside any large dense network trained with SGD, there exists a small sub-network (~ 10 – 20% of weights) that, when trained *from scratch* with the same initialization, reaches the same accuracy as the full dense network.

Implication. The optimizer is not finding the only good solution — it's finding one of many embedded in a vastly over-parameterized landscape. Optimization in deep learning is more about *which* good solution you find than *whether* you find one.

Implicit bias of SGD. Two networks with the same training loss can generalize very differently. SGD has a *preference* among the many minima: it prefers solutions with small implicit norm, in some sense.

- Soudry, Hoffer, Nacson, Gunasekar, Srebro (2018), “The implicit bias of gradient descent on separable data”: for logistic regression, GD converges to the max-margin direction even though no margin term appears in the loss.
- Gunasekar et al. (2018), “Implicit Bias of Gradient Descent on Linear Convolutional Networks”: different network parameterizations give different implicit norms.
- Belkin, Hsu, Ma, Mandal (2019), “Reconciling modern machine learning practice and the bias-variance trade-off”: the *double descent* curve — generalization improves *past* interpolation.

Why this matters for optimization. The choice of optimizer changes which minimum you converge to. Adam's minima are systematically different from SGD's minima — and the difference shows up in test accuracy. *Optimization and generalization are coupled in deep learning.*

Open frontier. Why does over-parameterization help? Why does early stopping help? Why do sharper minima generalize worse? Each of these has multiple proposed answers and no definitive one. Active research in 2024–2026.

Roadmap

From Yesterday's Theory to Today's Practice

The DL Training Problem

ADMM is not Adam

Per-coordinate Adaptive Methods

When Adam Fails, and Why

Second-Order Revival

Loss-Landscape Phenomena

Wrap-Up

The optimizer choice in 2025: what to use when

Setting	Default optimizer
Vision classifier (ResNet-50 or smaller)	SGD with momentum + cosine LR
Vision transformer / large CNN	AdamW
Language model fine-tuning	AdamW
Foundation-model pretraining (1B+ params)	AdamW or Sophia, with LR warmup
Reinforcement learning (policy gradient)	Adam or PPO + Adam
Convex problem ($\leq 10^5$ params)	L-BFGS (you're not in DL)
Convex problem with constraints	IPOPT (you're not in DL)

Three rules of thumb.

1. Default to AdamW with $\eta = 10^{-3}$, $(\beta_1, \beta_2) = (0.9, 0.999)$, $\lambda = 0.01$. Adjust η first.
2. Add learning-rate warmup (linear ramp from 0 to η over $\sim 2,000$ steps) for any model with $> 100\text{M}$ parameters.
3. For large-batch training, apply the linear-scaling rule and watch the loss curve for divergence near the critical batch size.

What's missing from this lecture.

- Distributed / federated optimization (Lecture 4 splitting; production: ZeRO, FSDP, ring-allreduce).
- Mixed-precision training and gradient scaling.
- Schedule-Free SGD (Defazio–Yaida 2024) — recent alternative to LR schedules.
- Optimization for fine-tuning: LoRA, prefix tuning, parameter-efficient fine-tuning (PEFT).

Bridge to next lecture: RLHF

Where today leaves off. We can train a foundation model (a language or vision model) using AdamW or Sophia on standard cross-entropy loss. The result is a *pretrained* model: it speaks fluent English, knows facts, can write code.

What's still missing. The pretrained model isn't *aligned* with what humans want. It hallucinates, ignores instructions, produces toxic output, and makes basic reasoning mistakes. Pretraining on the web doesn't fix any of this.

The optimization problem of alignment. Given:

- A pretrained model π_{θ_0} .
- A reward model $r_\phi : (\text{prompt}, \text{response}) \rightarrow \mathbb{R}$ trained on human-preference data.
- A reference model (usually a copy of π_{θ_0}).

solve

$$\max_{\theta} \mathbb{E}_{\text{prompt} \sim \mathcal{D}, \text{response} \sim \pi_{\theta}} [r_{\phi}(\text{prompt}, \text{response})] - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\theta_0}).$$

This is an **RL problem with a learned reward**. *No closed-form gradient with respect to θ* because the response is sampled.

Algorithms.

- **PPO** (proximal policy optimization, Schulman et al. 2017) — the original RLHF optimizer.
- **DPO** (direct preference optimization, Rafailov et al. 2023) — closed-form alternative; no reward model needed.
- **GRPO** (group relative policy optimization, DeepSeek 2024) — the algorithm behind DeepSeek-V3 and R1.

Tuesday, May 5: we'll derive each of these from the same KL-regularized expected-reward objective above, and see how they relate to the SGD / Adam / Nesterov story we've built so far.

Pre-class reading. Stiennon et al. 2020 ("Learning to summarize with human feedback", NeurIPS) for the original RLHF, and Rafailov et al. 2023 ("Direct Preference Optimization", NeurIPS) for the recent shift.

References

Surveys and textbooks.

- **Bishop & Bishop** (2024), *Deep Learning: Foundations and Concepts*, Springer.
The textbook companion to today's lecture. Ch. 6 (deep nets), Ch. 7 (gradient descent, RMSProp, Adam — Algorithm 7.4), Ch. 8 (backpropagation — esp. §8.1.2 for general feed-forward), Ch. 9 (regularization and inductive bias).
- Bottou, Curtis, Nocedal (2018), *Optimization Methods for Large-Scale Machine Learning*, SIAM Review 60(2). The optimization-side survey, with mini-batch theory and variance-reduced methods.
- Goh (2017), *Why Momentum Really Works*, Distill. The geometric picture (today's opening video).
- Ruder (2016), *An overview of gradient descent optimization algorithms*, arXiv:1609.04747. Compact survey of every optimizer covered today.

Algorithm papers.

- Duchi, Hazan, Singer (2011), *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*, JMLR 12 (AdaGrad).
- Kingma, Ba (2015), *Adam: A Method for Stochastic Optimization*, ICLR.
- Loshchilov, Hutter (2019), *Decoupled Weight Decay Regularization*, ICLR (AdamW).
- Reddi, Kale, Kumar (2018), *On the Convergence of Adam and Beyond*, ICLR best paper.
- Martens, Grosse (2015), *Optimizing Neural Networks with Kronecker-factored Approximate Curvature*, ICML (K-FAC).
- Gupta, Koren, Singer (2018), *Shampoo: Preconditioned Stochastic Tensor Optimization*, ICML.
- Liu, Ren, Tian, Liang, Tang (2023), *Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training*.

Loss-landscape papers.

- Dauphin, Pascanu, Gulcehre, Cho, Ganguli, Bengio (2014), *Identifying and attacking the saddle point problem in high-dimensional non-convex optimization*, NeurIPS.
- Cohen et al. (2021), *Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability*, ICLR 2022.
- Frankle, Carbin (2019), *The Lottery Ticket Hypothesis*, ICLR.
- Goyal et al. (2017), *Accurate, Large Minibatch SGD*.
- McCandlish, Kaplan, Amodei (2018), *An Empirical Model of Large-Batch Training*.